



主存储器

基本概念

存储器分类

1. **随机存储器**(random access memory, RAM): 可以随机读写
2. **只读存储器**(read only memory,简称ROM): 只能读不能写, 制造芯片时预先写入内容
3. **可程序的只读存储器**(programmable ROM, 简称PROM): 一次性写入, 写入后只读
4. **可擦除可程序只读存储器** (erasable PROM, 简称EPROM) : 可用紫外线擦除
5. **可用电擦除的可编程只读存储器** (electrically EPROM, 简称E2PROM) : 可用电擦除

RAM为易失性存储器, 其他为非易失性存储器

主要技术指标

1. 寻址空间
 - 最小信息单位是**存储字**
 - 地址线位数决定可直接寻址的最大空间
2. 编址方式
 - 字编址: 以字为单元, 字可以是2/4字节
 - 字节编址: 以字节为单元
3. 速度
 - 存取时间: 从启动一次存储器操作到完成该操作所经历的时间
 - 存储周期: 连续启动两次独立的存储器操作(例如连续两次读操作)所需间隔的最小时间

基本操作

- **地址寄存器AR**: k位字长, 则允许主存包含 2^k 个可寻址单位 (字节/字)
- **数据寄存器DR**: n位字长, CPU和主存之间通过总线进行n位数据传送

读操作

1. CPU把数据存放地址送AR, 经地址总线送主存
2. CPU通过控制线发read请求, wait MFC

3. 数据通过数据总线送DR后，主存发MFC (ready) 信号

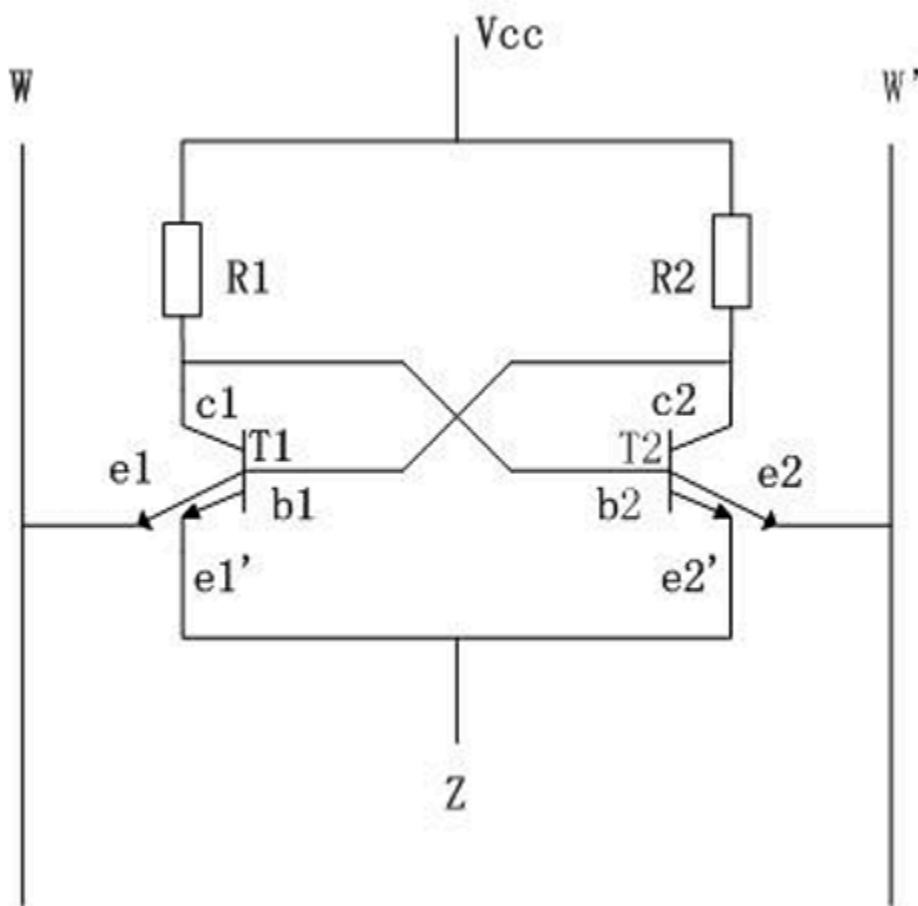
写操作

1. CPU把数据存放地址送AR，经地址总线送主存，数据送DR
2. CPU通过控制线发write请求，wait MFC
3. 数据从DR通过数据总线送主存后，主存发MFC (ready) 信号

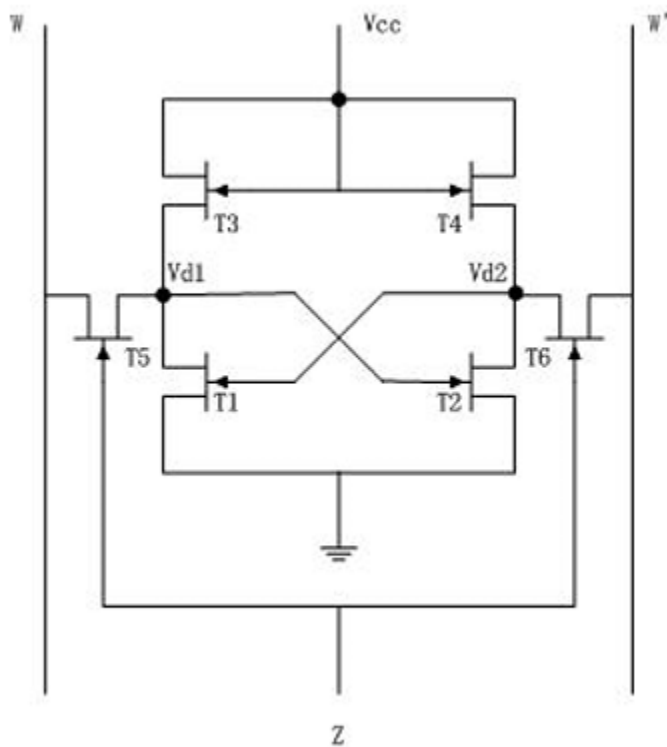
随机存储器RAM

双极型随机存储器

1. 基本存储单元
 - 电流向电压小的方向流



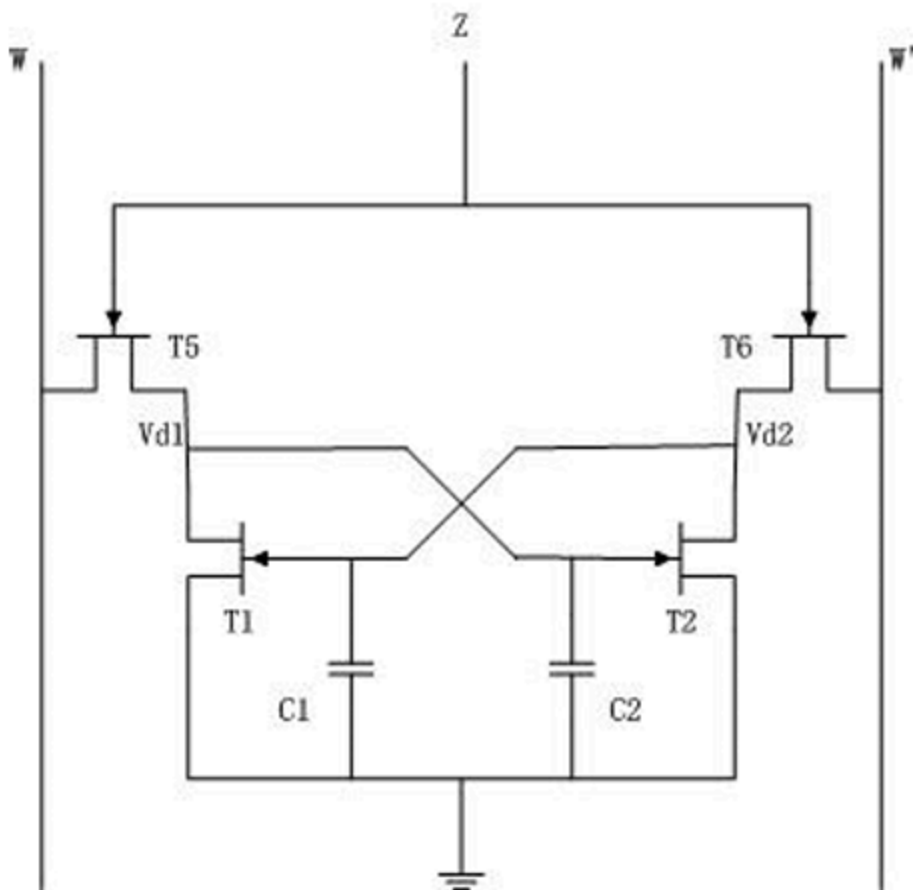
- T1和T2构成双稳态电路
 - T1导通，T2截止为“1”
 - T1截止，T2导通为“0”



- T1和T2构成双稳态电路
 - T1导通, T2截止为“0”
 - T1截止, T2导通为“1”
 - T3和T4为负载管
 - T5和T6为门控管
 - W、W'为读写线
 - Z为选通线
2. 保持状态: $V_z=0V$, T5、T6截止
 3. 读操作: $V_z=5V$, T5、T6导通
 - 读“1”: T2导通, V_{d1} 高电位通过T5传到W上, W有电流
 - 读“0”: T1导通, V_{d2} 高电位通过T6传到W'上, W'有电流
 4. 写操作: $V_z=5V$, T5、T6导通
 - 写“1”: $V_w=5V$, $V_{w'}=0V$,使得T1截止, T2导通
 - 写“0”: $V_w=0V$, $V_{w'}=5V$,使得T2截止, T1导通
 5. 特点
 - 不需要刷新外围电路
 - 结构简单, 可靠性高
 - 集成度低, 功耗高

四管动态存储器

1. 基本存储单元



- 使用C1、C2存储电荷来代表二进制信息
- C1高电位时T1导通
- C2高电位时T2导通
- T1导通, T2截止为“0”
- T1截止, T2导通为“1”

2. 读操作: Vz高电位, T5、T6导通

- 读“1”: T2导通, C2高电位通过T5传到W上, W有电流
- 读“0”: T1导通, C1高电位通过T6传到W'上, W'有电流

3. 写操作: Vz高电位, T5、T6导通

- 写“1”: Vw高电位, Vw'低电位,使得T1截止, T2导通, 给C2充电、C1放电
- 写“0”: Vw高电位, Vw'低电位,使得T2截止, T1导通, 给C2放电、C1充电

4. 保持状态: Vz低电位, T5、T6截止

- 此时C1、C2会缓慢漏电, 约2ms会漏完
- 因此每隔2ms需要刷新一次

5. 刷新过程: Vz高电位, T5、T6导通

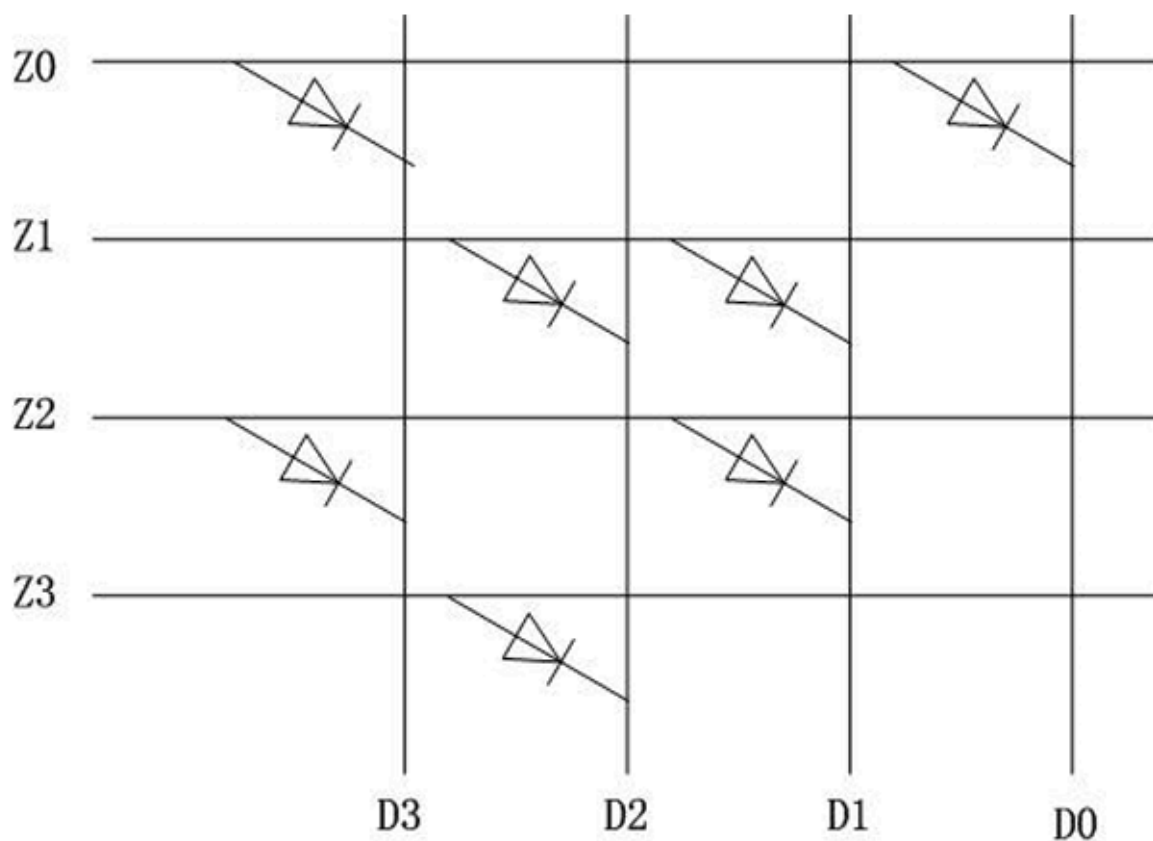
- Vw、Vw'均为高电位, 同时给C1、C2充电
- 充电相同时间, 使得原本高电位保持高电位, 原本低电位保持低电位

6. 特点

- 集成度高，功耗低
- 外围电路复杂，在刷新时不能读写

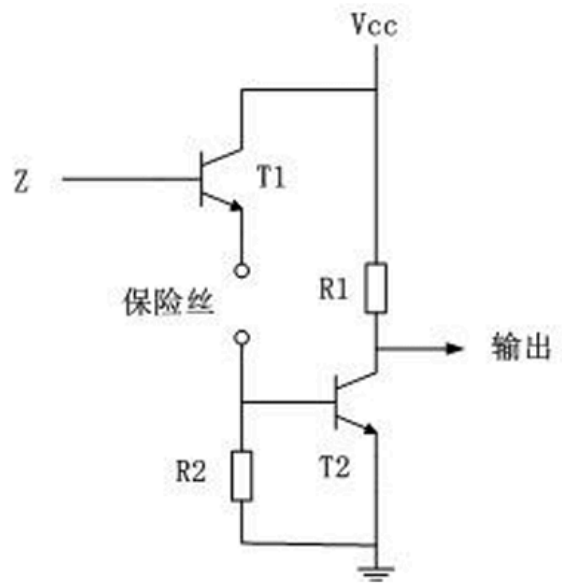
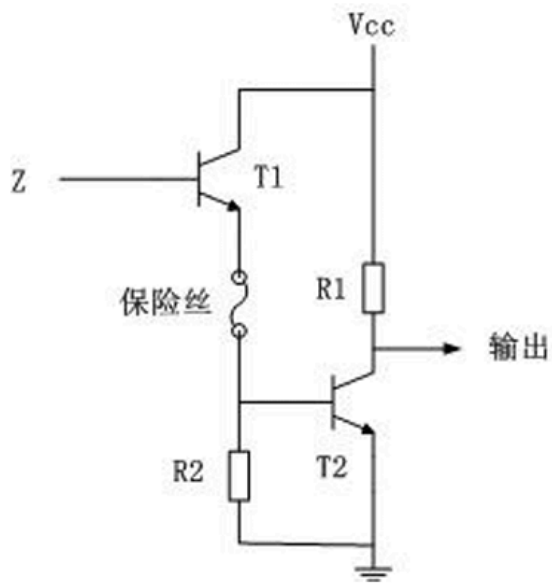
只读存储器

只读存储器ROM



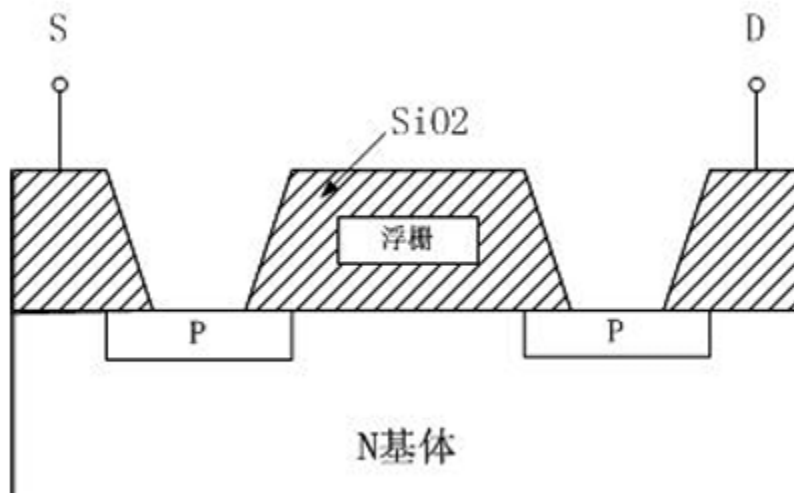
PROM

- 刚出厂时，熔丝全部接通
- 用户根据自己需要写入数据（断开某些单元的熔丝）



EPROM

- 基本存储单元由一个管子组成，但与其他电路相比管子内多增加了一个浮置栅



E2PROM

- E2PROM的编程原理与EPROM相同，但擦除原理完全不同，重复改写的次数有限制
- 栅极氧化层比EPROM薄，因此具有电擦除功能

Flash Memory

- 与E2PROM相同之处是用电来擦除
- 但是它只能擦除整个区或整个器件

半导体存储系统的组成

存储容量扩展

1. 位扩展：对数据字长进行扩展，串行连接
2. 字扩展：对地址线宽度进行扩展，并行连接
3. 字位同时扩展：先位后字

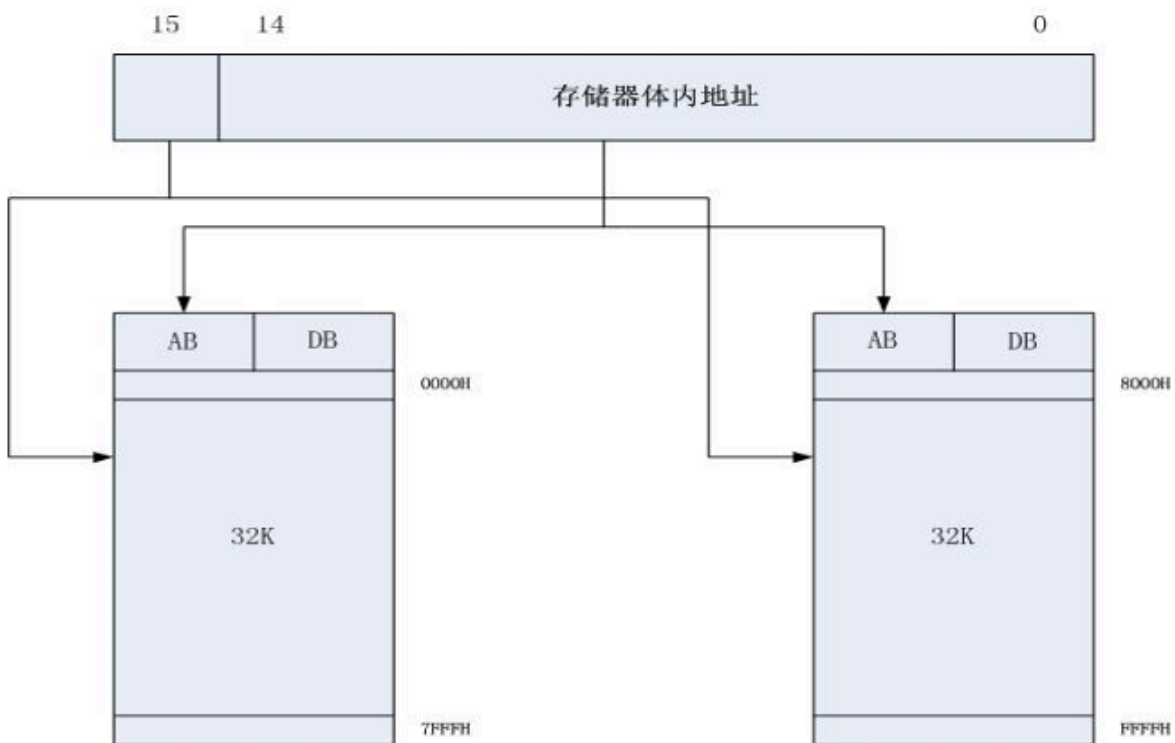
存储控制

- 地址多路转换线路
- 地址选通
- 刷新逻辑
 - 集中刷新
 - 在一个刷新周期内，利用一段固定的时间，依次对存储器的所有行逐一再生，在此期间停止对存储器的读和写
 - 缺点是在刷新期间不能访问存储器，有时会影响计算机系统的正确工作
 - 分布式刷新
 - 采取在2ms时间内分散地将1024行刷新一遍的方法
- 读/写控制逻辑

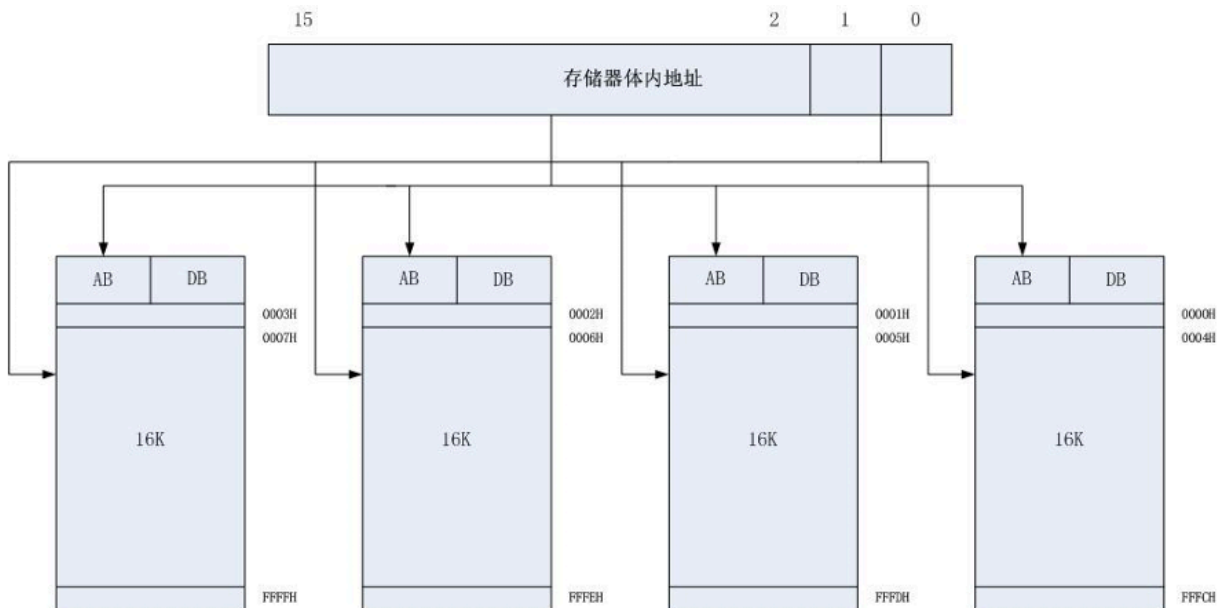
多存储体存储器

地址划分方法

1. 按高位地址划分



2. 按低位地址划分



- 模块数必须是2的整数幂，否则，地址不连续
- 任一模块失效就会造成地址空间的缺陷而导致整个程序无法运行
- 不利于存储器模块数的增量式扩展

访问控制方法

1. 同时访问

- 所有模块同时启动一次存储周期，相对各自的数据寄存器并行地读出或写入信息

- 一次提供多个数据或多条指令，比较适合用于对多数据流或多指令流进行并行处理
2. 交叉访问
 - M个模块按一定的顺序轮流启动各自的访问周期，启动两个相邻模块的最小时间间隔等于单模块访问周期的1/M
 - 适合用于以流水线方式向中央处理器提供指令和数据

高速缓冲存储器Cache

工作原理

- 程序访问的局部化性质：在一个较短的时间间隔内，CPU访问存储器的地址往往集中在存储器逻辑地址空间的很小范围内
 - **命中率**=访问信息在Cache中的次数/访问Cache的总次数
 - 存储容量太小会使命中率太低
 - 太大会增加成本
1. 读操作
 - 数据在Cache中，直接读
 - 数据不在Cache中，把数据所在字块从主存调入；两种方案：
 - 全部送入Cache，再从Cache读
 - 送入Cache的同时，数据直接送CPU（**通过式读**）
 2. 写操作，两种方式：
 - **写回法**
 - 只向Cache写，标志为脏页；脏页被替换时写回主存
 - 速度快
 - 但主存中的字块未经即时修改而失效
 - **通过式写**
 - 向Cache写的同时，也向主存写；Cache和主存随时保持一致
 - 保持主存数据正确性
 - 但可能增加不必要写入
 - 被写单元不在Cache中，则直接写入主存
- 平均存取时间= $h \times T_c + (1-h) \times (T_c + T_m) = T_c + (1-h) \times T_m$
 - h：命中率
 - T_c ：Cache的存取时间

- T_m : 主存的存取时间

地址映像

对Cache存取时，把主存地址按地址映像转换为Cache地址；作如下假设

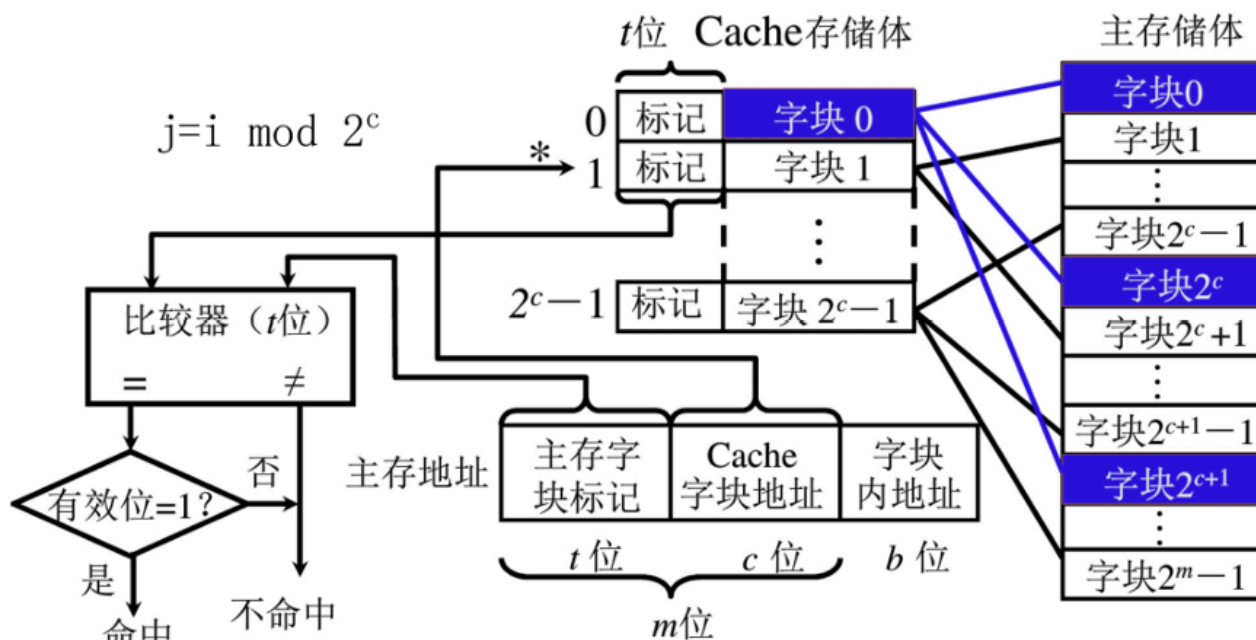
- 主存空间被分为 2^m 个块
- Cache空间被分为 2^c 个块
- 每个块大小为 2^b 字节
- i 为主存字块号

直接映像

Cache字块号 $j = i \bmod 2^c$

标记位($m - c$ 位) $= i \div 2^c$

- 根据主存地址计算Cache字块号和标记位值，取出Cache中该字块号的标记位并比较

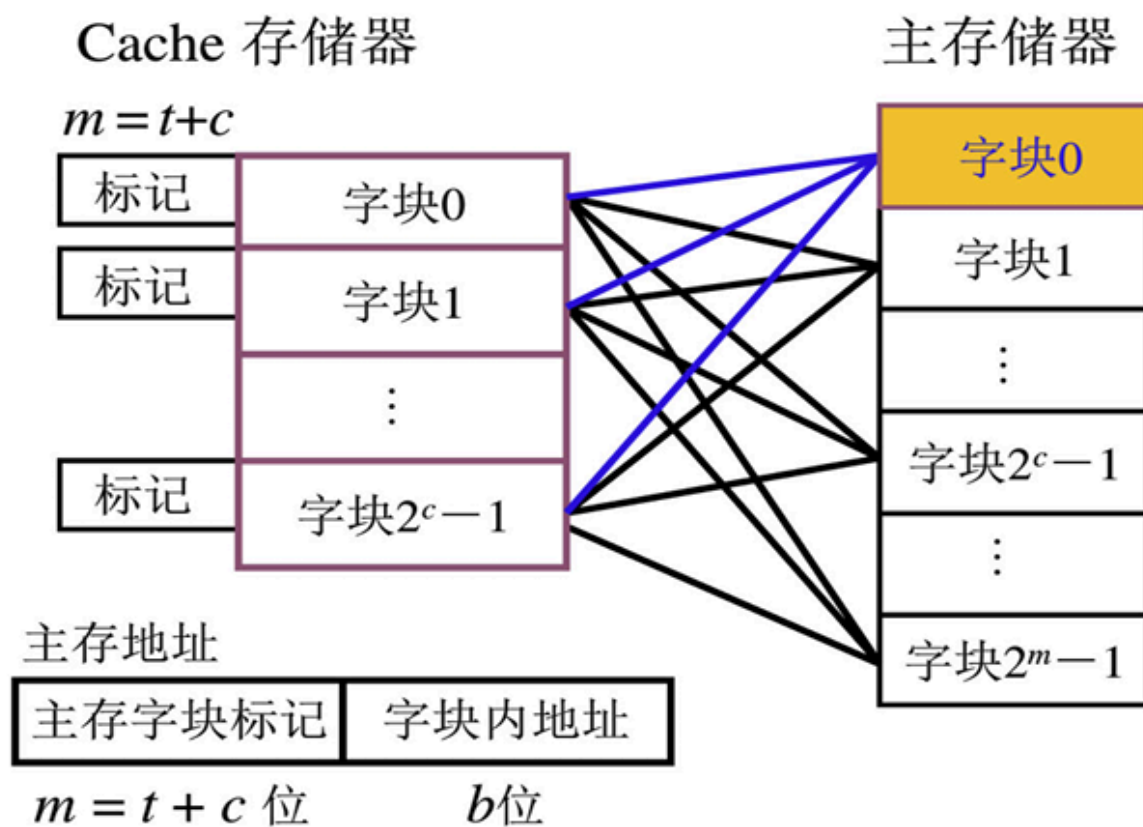


- 主存中字块对应到Cache中为固定字块
- 优点：实现简单
- 缺点：对Cache利用率低，可能出现Cache未满但需要替换块

全相联映像

标记位(m 位) $= i$

- 遍历Cache的所有字块，比较标记位



- 主存中字块可以放在Cache中的任意字块
- 优点：灵活
- 缺点：成本高

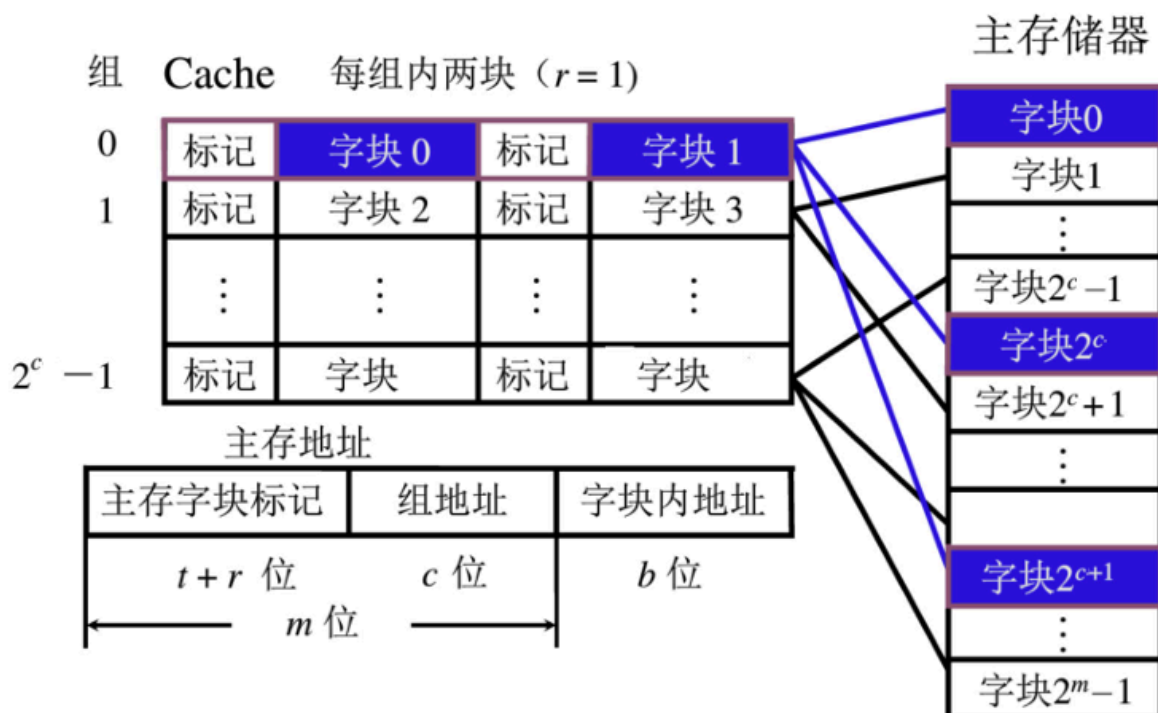
组相联映像

- Cache的 2^c 个块按每 2^r 个块一组被分为 2^{c-r} 组

Cache组号 $j = i \bmod 2^{c-r}$

标记位($m - c$ 位) $= i \div 2^{c-r}$

- 根据主存地址计算Cache组号和标记位，遍历Cache中该组的所有字块，比较标记位



- 主存中字块对应到Cache中为固定组的任意位置
- 直接映像和全相联映像方式的一种折衷方案

替换算法

当主存字块需要调入Cache中而它的可用位置被占满时，计算哪个块被替换掉的算法

FIFO(First In First Out)

- 把最先调入Cache的字块替换出去
- 不需要记录每个字块的使用情况
- 实现容易、开销小

LRU(Last Recently Used)

- 把近期最少使用的字块替换出去
- 需要记录每个字块的使用情况
 - 当某个块a命中时
 - 块a计数值清零
 - 计数值低于a原本计数值的块计数值+1
 - 计数值高于或等于a原本计数值的块计数值不变
 - 未命中而调入新块，且分组未滿时

- 被调入的块计数值为0
- 其他块计数值+1
- 未命中而调入新块，且分组已满需要替换时
 - 计数值最大的块被替换为新块，计数值为0
 - 其他块计数值+1

虚拟存储器

程序员编程序使用虚拟地址

程序运行时通过虚拟地址到主存中查询真实地址

主存-辅存与Cache-主存的比较

1. 辅存/主存的访问时间比更大
2. 辅存/主存每次传送的基本信息单元更大（段或页）
3. 地址映像和替换策略的原理相同

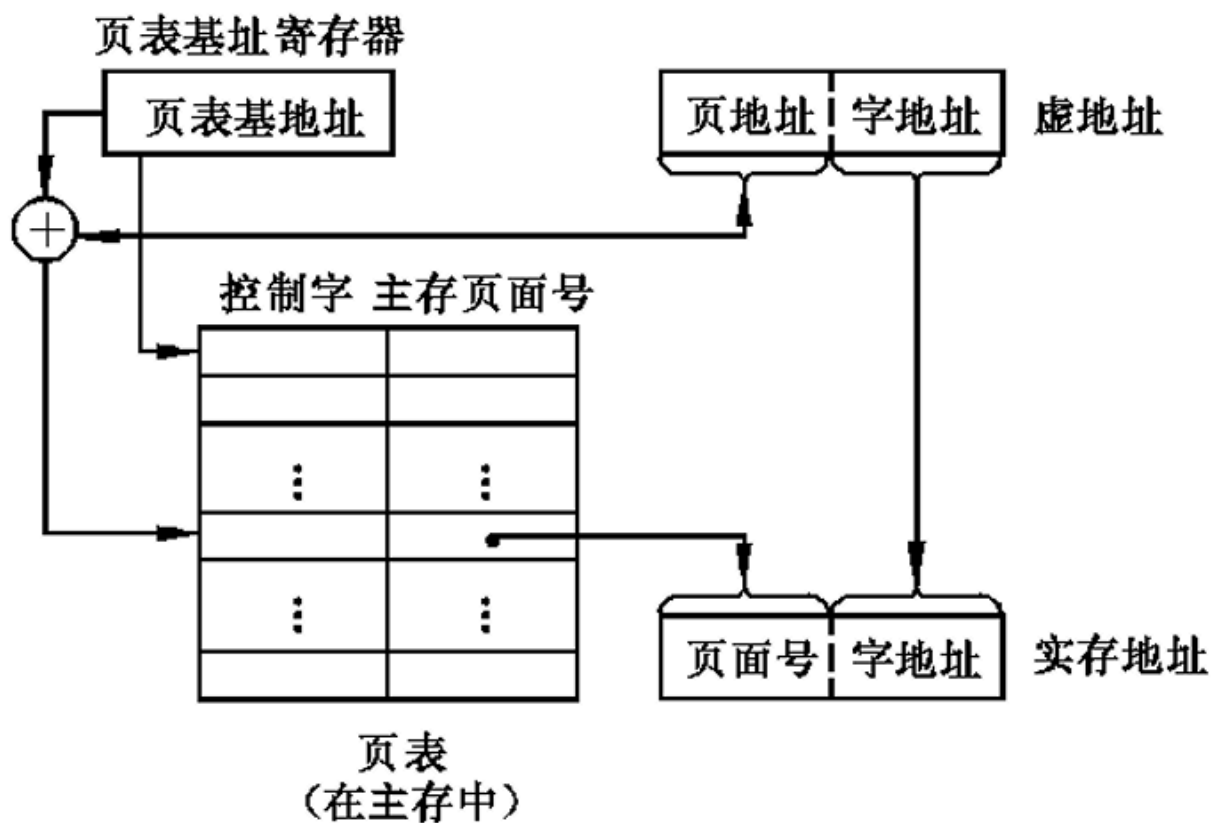
主存-辅存的信息传送单位和存储管理

1. 段存储
 - 按照程序的逻辑结构划分成的多个相对独立部分
 - 段的分界与程序的自然分界相对应
 - 段的逻辑独立性使它易于编译、管理、修改和保护，也便于多道程序共享
 - 但容易在段间留下许多空余的零碎存储空间不好利用，造成浪费
2. 页存储
 - 信息传送单位是定长的页，主存的物理空间也被划分为定长的页
 - 比段式管理系统的空间浪费要小得多
 - 由于页不是逻辑上独立的实体，所以处理、保护和共享都不及段式来得方便
3. 段页式存储
 - 程序按模块分段，段内再分页
 - 出入主存仍以页为信息传送单位
 - 用段表和页表(每段一个页表)进行两级管理

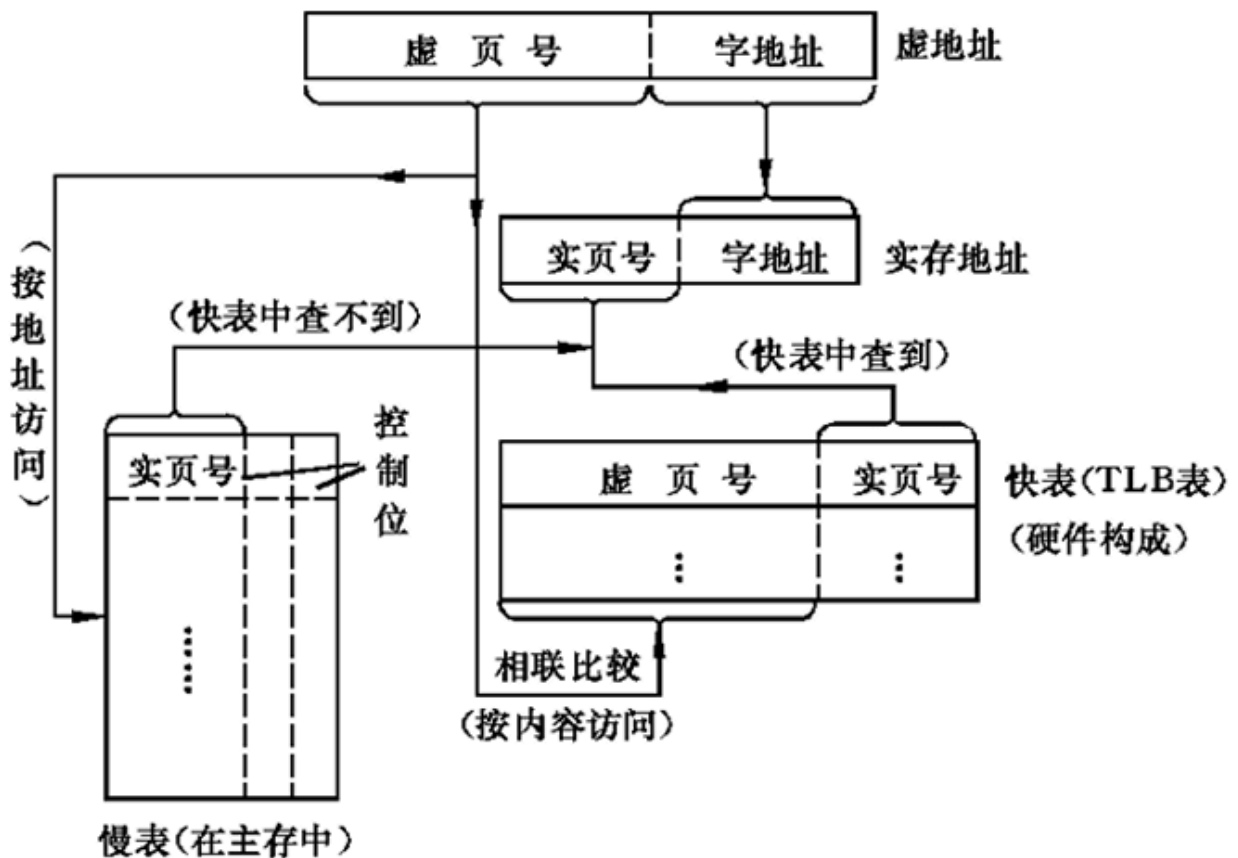
页式虚拟存储器

- 主存空间被分为 L 个页

- 虚拟空间被分为 M 个页
- 页的大小为2的整数幂个字
- 把虚拟地址分为两个字段
 - 高字段为虚页号
 - 低字段为页内字地址
- 虚拟地址到主存实地址的变换通过页表来实现



- 控制字包括：
 - 装入位：为“1”表示该虚页已经从辅存调入主存
 - 修改位：是否为脏页
 - 替换控制位：给LRU用
- 把页表的最活跃部分存放在快速存储器中组成快表以减少时间开销



段页式虚拟存储器

- 虚拟地址到主存实地址的变换通过一个段表和一组页表来实现
- 段表中的每个表目对应一个段，每个表目有一个指向该段的页表的起始地址(页号)及该段的控制保护信息
- 由页表指明该段各页在主存中的位置以及是否已装入、已修改等标志
- 如果有多个用户在机器上运行（多道程序），则虚拟地址应包括：基号D、段号S、页号P、页内地址d

